Performance Testing

Date	04 July 2026
Team ID	
Project Name	AI-Powered Debt Relief & Financial Recovery Platform
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman
Type of Testing	Load Testing
Target Module / API	User Authentication, AI Financial Analysis API, Debt Recommendation API
Test Environment	Local Flask Server
Test Date	04-07-2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	User Login Authentication	20	60	Login should succeed with quick response.
2	AI Financial Analysis	15	60	AI should generate financial analysis successfully.
3	Debt Repayment Recommendation	15	60	Personalized repayment plan should be generated.
4	Dashboard Data Retrieval	20	60	Dashboard should load financial reports without delay.

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.3 sec	Pass	Fast response
2	Response Time (Max)	< 5 seconds	3.2 sec	Pass	Within acceptable limit
3	Throughput (Req/sec)	> 15 req/sec	18 req/sec	Pass	Stable throughput
4	Error Rate	< 1%	0.2%	Pass	Very low error rate
5	CPU Utilization	< 80%	55%	Pass	Normal utilization
6	Memory Utilization	< 80%	61%	Pass	Stable memory usage

Step 4: Observations & Analysis

Key Findings

- The application handled multiple requests efficiently with low response times.
- AI-generated financial analysis and repayment recommendations were delivered successfully.
- No significant performance degradation was observed during testing.

Bottlenecks Identified

- Slight increase in response time during AI processing under higher load.
- External AI API response time may vary depending on network conditions.

Optimization Steps Taken

- Optimized database queries for faster data retrieval.
- Reduced unnecessary API calls.
- Improved backend processing and response handling.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):